GestLab — Avance del proyecto

Diseño Orientado a Objetos · Momento Evaluativo 2

Proyecto	GestLab: sistema web para la gestión y el análisis de la carga laboral de operarios
Integrantes	Pedro David González Lesmes · *segundo integrante: por completar*
Fecha	Octubre de 2026
Repositorio	https://github.com/PedroGonzalez9612/TaskManager

GestLab responde tres preguntas de una empresa con operarios: qué se debe hacer, quién lo está haciendo y cuánto trabajo hay en curso. Tiene tres roles: el Superadmin registra empresas, el Administrador crea y asigna actividades, y el Operario las ejecuta.

Este documento sigue los cinco puntos del enunciado, en el mismo orden.

---

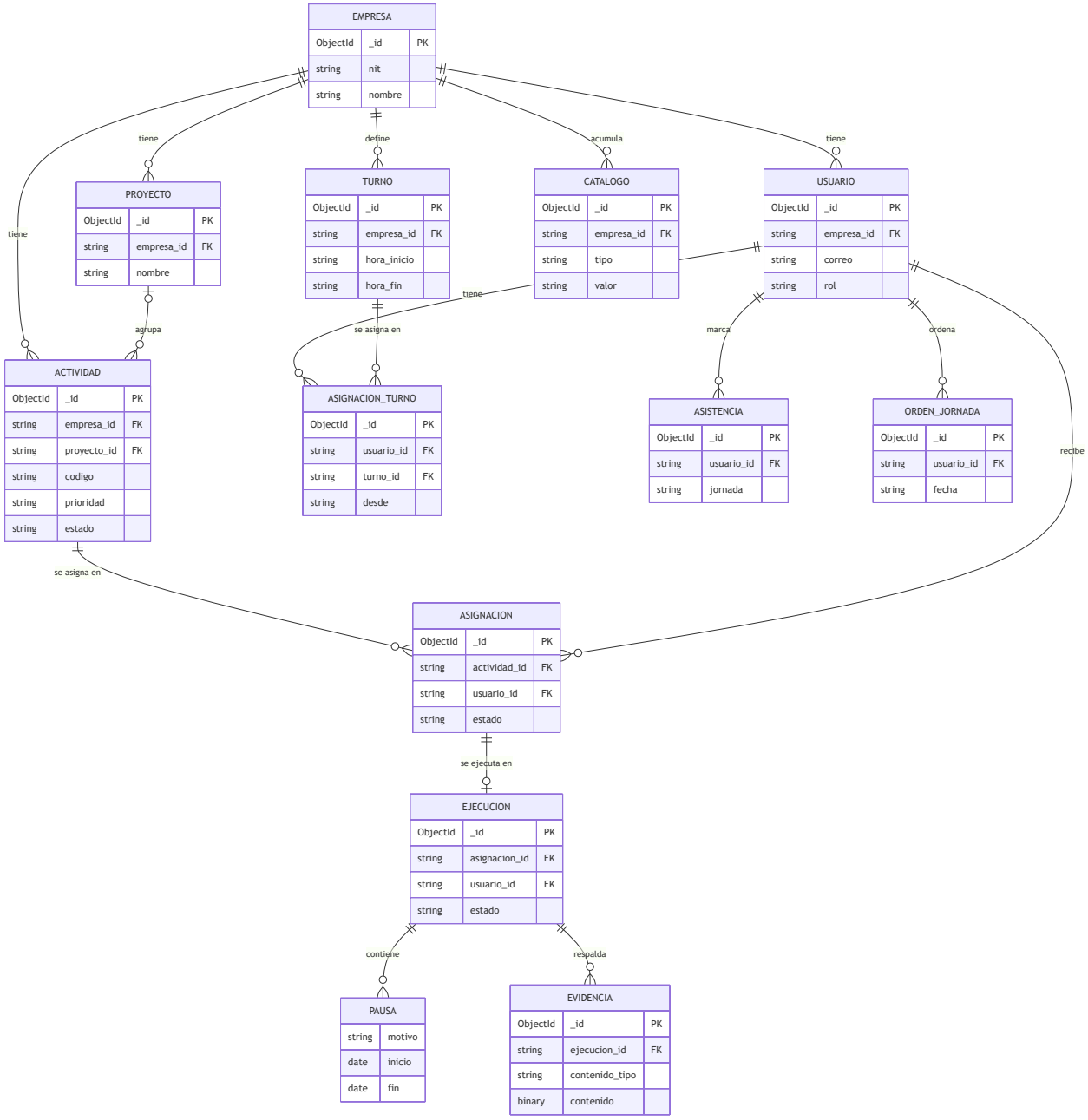
1. Modelado de la base de datos

El motor es **MongoDB 7**, una base de datos de documentos. Por eso el modelo lógico se expresa en colecciones y documentos, y no en tablas.

1.1 Modelo lógico

Tres decisiones definen el modelo:

- **Todo pertenece a una empresa.** Casi todas las colecciones llevan `empresa_id`, y cada consulta filtra por él: una empresa nunca ve datos de otra.
- **La ejecución pertenece a la asignación.** Una actividad puede tener varios operarios. Cada uno tiene su propia ejecución, con su tiempo, sus pausas y su observación. La actividad termina cuando todos finalizan.
- **Nada se borra.** Una asignación se cierra (devuelta o retirada) y una actividad se cancela con motivo. El historial queda en la base.



Cómo leer las relaciones

Relación	Regla
Empresa – Usuario	Una empresa tiene varios usuarios. El Superadmin no pertenece a ninguna
Proyecto – Actividad	Una actividad es independiente o pertenece a un solo proyecto
Actividad – Asignación – Usuario	Relación de muchos a muchos: una actividad se asigna a uno o varios operarios
Asignación – Ejecución	Cada asignación tiene como máximo una ejecución, que nace cuando el operario inicia

Relación	Regla
Ejecución – Pausa	Las pausas viven dentro de la ejecución; no existen por separado
Ejecución – Evidencia	Cada ejecución tiene hasta 5 archivos de evidencia (fotos o PDF). Van en su propia colección porque pesan mucho más que el resto de la ejecución
Usuario – Turno	Un operario tiene un solo turno vigente; se guarda la fecha de cada cambio
Usuario – Asistencia	Una asistencia por operario y jornada

Referenciar o incrustar. En una base de documentos, cada relación se resuelve de una de dos formas:

- **Referencia** (se guarda el identificador del otro documento): cuando el otro dato tiene vida propia y se consulta por separado. Así se relacionan empresas, usuarios, actividades, asignaciones y ejecuciones.
- **Documento incrustado** (el dato va dentro del documento principal): cuando el dato no tiene sentido sin su dueño y siempre se lee con él. Así se guardan las pausas de una ejecución, las reprogramaciones y la cancelación de una actividad, la devolución de una asignación y la revisión de una asistencia.

1.2 Modelo físico

La base se llama `taskmanager` y tiene 13 colecciones. Los índices los crea la aplicación al arrancar, desde cada repositorio (`app/repositories/`).

Convenciones de tipos

- Los identificadores propios son `ObjectId`. Las referencias a otros documentos se guardan como texto (`string`) con ese mismo identificador.
- Los momentos (inicio, fin, marca de entrada) son `date` en UTC, y los pone el servidor.
- Los días (`fecha_programada`, `jornada`) son texto `AAAA-MM-DD`, y las horas del día son texto `HH:MM`: un día de trabajo no es un instante, depende de la zona horaria de la empresa.
- Los estados, roles y prioridades son texto con los valores de las enumeraciones de `app/models/enums.py`.

`empresas`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`nombre`, `nit`, `sector`, `direccion`	string	Datos de la empresa
`limite_administradores`, `limite_operarios`	number	Cuántos usuarios de cada rol puede tener
`zona_horaria`	string	Define qué día es "hoy" para la empresa

Campo	Tipo	Descripción
`logo`	binario	Imagen del logo (marca blanca)
`logo_tipo`	string	Tipo de la imagen (PNG, JPG o WEBP)
`consecutivo_actividad`	number	Último número usado en el código de las actividades

`usuarios`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`nombre`, `correo`	string	Datos del usuario
`contraseña_hash`	string	Contraseña cifrada con bcrypt; nunca sale en la API
`rol`	string	`SUPERADMIN`, `ADMINISTRADOR` u `OPERARIO`
`empresa_id`	string o null	Empresa a la que pertenece; `null` en el Superadmin

`proyectos`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`nombre`, `descripcion`	string	Datos del proyecto
`empresa_id`	string	Empresa dueña
`creado_por_id`	string	Administrador que lo creó
`fecha_creacion`	date	Momento de creación

Índice: `empresa` (`empresa_id`).

`actividades`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`codigo`	string	Código visible, por ejemplo `OT-0001`
`titulo`, `descripcion`, `ubicacion`	string	Datos de la actividad
`empresa_id`	string	Empresa dueña

Campo	Tipo	Descripción
`proyecto_id`	string o null	Proyecto al que pertenece; `null` si es independiente
`categoria`	string	Producción, mantenimiento, calidad, limpieza o logística
`prioridad`	string	`BAJA`, `MEDIA`, `ALTA` o `URGENTE`
`tiempo_estimado_min`	number	Tiempo estimado de la actividad completa, en minutos
`fecha_programada`	string	Día en que debe hacerse
`hora_programada`	string o null	Hora fija; `null` si es de horario flexible
`fecha_original`	string	Primer día programado, para medir los días de retraso
`estado`	string	Estado calculado a partir de las ejecuciones de sus operarios
`reprogramaciones`	lista de documentos	Cada paso automático a otra jornada: origen, destino y fecha
`cancelacion`	documento o null	Motivo, autor y fecha, si fue cancelada
`fecha_no_realizada`	date o null	Cuándo quedó como No realizada
`requerimiento_id`	string o null	Campo heredado; se retira con los requerimientos

Índices: `empresa_fecha` (`empresa_id`, `fecha_programada`) y `empresa_proyecto` (`empresa_id`, `proyecto_id`).

`asignaciones`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`actividad_id`, `usuario_id`	string	Qué actividad y qué operario
`estado`	string	`ACTIVA`, `DEVUELTA` o `RETIRADA`
`asignada_por_id`	string	Administrador que asignó
`fecha_asignacion`	date	Cuándo se asignó
`fecha_fin`	date o null	Cuándo se cerró
`retirada_por_id`	string o null	Administrador que la retiró
`devolucion`	documento o null	Motivo y fecha, si el operario la devolvió

Índices: `asignacion_activa_unica` (`actividad_id`, `usuario_id`), **único y parcial** sobre las activas; y `usuario_estado` (`usuario_id`, `estado`).

`ejecuciones`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`asignacion_id`	string	Asignación a la que pertenece
`actividad_id`, `usuario_id`	string	Copias para consultar sin pasar por la asignación
`estado`	string	`EN_PROGRESO`, `PAUSADA`, `FINALIZADA` o `CERRADA`
`fecha_inicio`, `fecha_fin`	date	Inicio y fin reales
`pausas`	lista de documentos	Cada pausa con su motivo, inicio y fin
`observacion`	string o null	Nota del operario al finalizar

Índices: `asignacion_unica` (`asignacion_id`), **único**; `una_en_progreso_por_usuario` (`usuario_id`), **único y parcial** sobre las que están en progreso; y `actividad` (`actividad_id`).

`turnos` y `asignaciones_turno`

Colección	Campo	Tipo	Descripción
`turnos`	`nombre`	string	Nombre del turno
	`hora_inicio`, `hora_fin`	string	Horario del turno
	`descanso_min`	number	Minutos de descanso; se restan de la capacidad
	`empresa_id`	string	Empresa dueña
`asignaciones_turno`	`usuario_id`, `turno_id`	string	Qué operario y qué turno
	`desde`	string	Día desde el que rige
	`asignado_por_id`	string	Administrador que lo asignó
	`fecha_registro`	date	Cuándo se registró el cambio

Índices: `empresa_hora_inicio` (`empresa_id`, `hora_inicio`) en `turnos`; `usuario_desde` (`usuario_id`, `desde`) en `asignaciones_turno`.

`asistencias`

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador

Campo	Tipo	Descripción
`usuario_id`, `empresa_id`	string	Operario y empresa
`jornada`	string	Día en que inicia el turno
`entrada`, `salida`	date	Marcas del operario; `salida` es `null` mientras no marque
`estado`	string	Por validar, validada o corregida
`revision`	documento o null	Quién validó o corrigió, cuándo, el motivo y las marcas originales

Índices: `una_por_jornada` (`usuario_id`, `jornada`), **único**; y `empresa_jornada` (`empresa_id`, `jornada`).

`ordenes_jornada`

Campo	Tipo	Descripción
`usuario_id`	string	Operario
`fecha`	string	Jornada
`actividad_ids`	lista de string	Orden que el operario eligió para sus actividades de horario flexible
`fecha_actualizacion`	date	Último cambio

Índice: `orden_unico_por_dia` (`usuario_id`, `fecha`), **único**.

`evidencias`

Fotos y adjuntos que el operario deja al finalizar una actividad. El archivo se guarda dentro del documento, como el logo de la empresa: con el límite de 5 MB por archivo cabe sin pasar del máximo de MongoDB (16 MB por documento).

Campo	Tipo	Descripción
`_id`	ObjectId	Identificador
`ejecucion_id`, `actividad_id`, `usuario_id`, `empresa_id`	string	A qué ejecución pertenece, y copias para comprobar quién la puede ver
`nombre`	string	Nombre del archivo, sin la carpeta de quien lo subió
`contenido_tipo`	string	`image/jpeg`, `image/png`, `image/webp` o `application/pdf`, reconocido por los primeros bytes del archivo y no por su nombre
`tamano`	number	Bytes del archivo
`fecha`	date	Cuándo se subió

Campo	Tipo	Descripción
`contenido`	binario	El archivo. Las consultas de listado no lo traen

Índice: `ejecucion` (`ejecucion_id`).

`catalogos`

Listas desplegables por empresa (motivos de pausa y de cancelación) que se llenan con lo que los usuarios escriben en "Otro".

Campo	Tipo	Descripción
`empresa_id`	string	Empresa dueña
`tipo`	string	`MOTIVO_PAUSA` o `MOTIVO_CANCELACION`
`valor`	string	Texto tal como se muestra
`clave`	string	El mismo texto normalizado, para no repetir valores

Índice: `valor_unico` (`empresa_id`, `tipo`, `clave`), **único**.

`requerimientos`

Colección de la primera versión (`titulo`, `descripcion`, `empresa_id`, `estado`, `fecha_creacion`). El modelo actual la reemplaza por proyectos y actividades independientes; se retira cuando la pantalla nueva del Administrador esté lista.

1.3 Reglas que garantiza la base de datos

Algunas reglas no pueden depender solo del código, porque dos peticiones pueden llegar al mismo tiempo. Para esas, la base rechaza el segundo intento con un índice único:

Regla de negocio	Índice
Un operario tiene una sola actividad en ejecución a la vez	`una_en_progreso_por_usuario`
Una asignación tiene una sola ejecución	`asignacion_unica`
Un operario no queda asignado dos veces a la misma actividad	`asignacion_activa_unica`
Una sola asistencia por operario y jornada	`una_por_jornada`
Un solo orden del día por operario y jornada	`orden_unico_por_dia`
Un motivo no se repite en el catálogo de una empresa	`valor_unico`

Los índices **parciales** aplican solo a los documentos que cumplen una condición. Por ejemplo, `una_en_progreso_por_usuario` solo mira las ejecuciones en progreso: un operario puede tener muchas ejecuciones finalizadas, pero solo una en curso.

1.4 Cómo comprobarlo en el motor

Con la aplicación levantada en Docker:

```
docker compose exec mongo mongosh taskmanager
```

```
show collections                    // las colecciones
db.ejecuciones.getIndexes()         // los índices de una colección
db.actividades.findOne()            // un documento de ejemplo
```

---

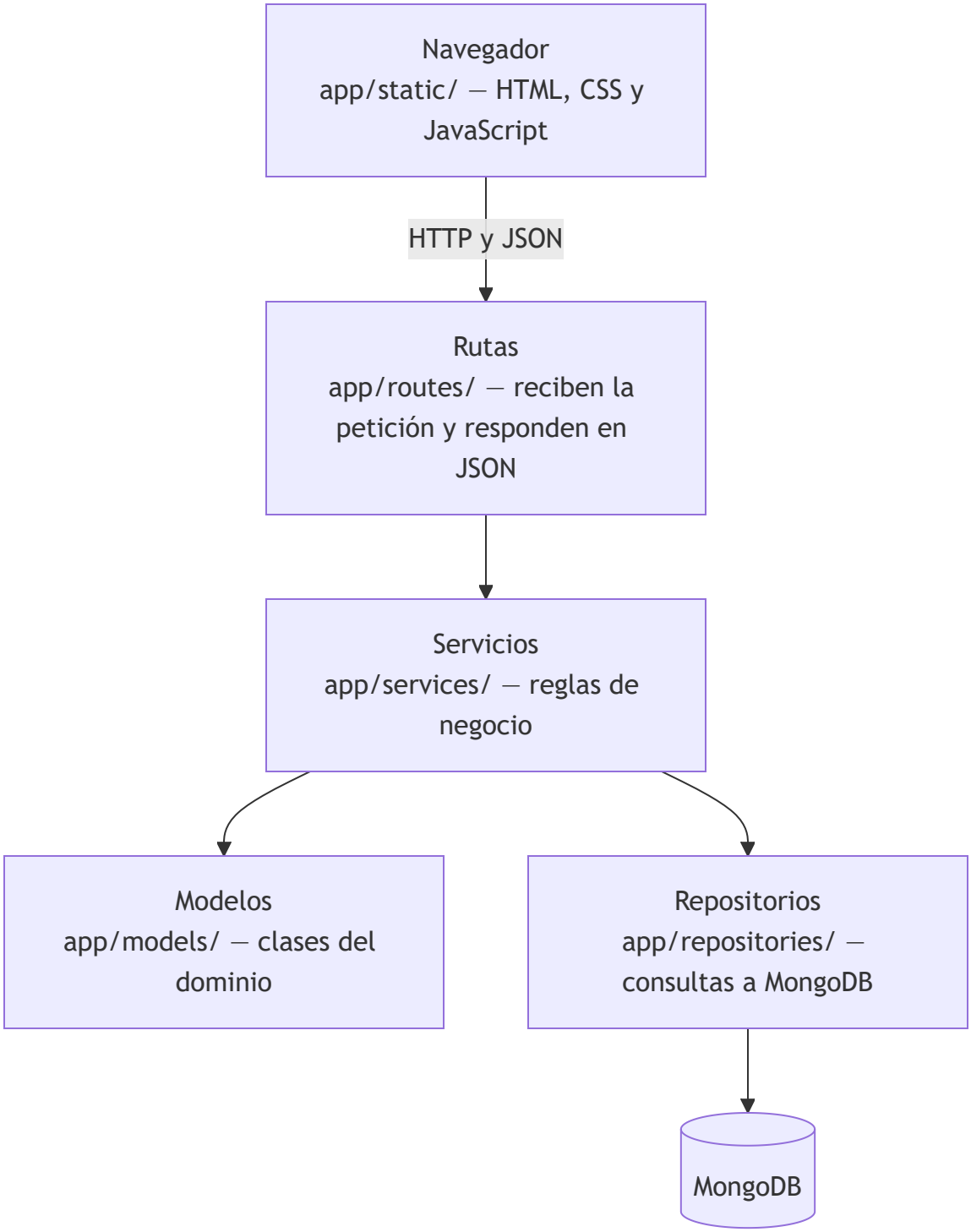
2. Inicio de la implementación

2.1 Tecnología web

Parte	Tecnología
Servidor	Python 3.11 con Flask: expone una API REST en JSON
Cliente	HTML, CSS y JavaScript en el navegador; consume la API con `fetch`
Base de datos	MongoDB 7, con la biblioteca `pymongo`
Seguridad	Sesión de Flask con cookie; contraseñas con bcrypt
Entorno	Docker Compose: un contenedor para la aplicación y otro para la base
Calidad	Pruebas automáticas con pytest y análisis con SonarQube

2.2 Arquitectura en capas

Cada capa solo conoce a la que tiene debajo:



Capa	Responsabilidad	Lo que no hace
Modelos	Clases del dominio (`Actividad`, `Asignacion`, `Ejecucion`, `Pausa`). Validan sus propios cambios de estado	No acceden a la base ni a Flask

Capa	Responsabilidad	Lo que no hace
Repositorios	Único lugar con consultas a MongoDB (patrón Repository)	No tienen reglas de negocio
Servicios	Reglas de negocio. Reciben los repositorios por el constructor	No conocen HTTP ni escriben consultas
Rutas	Reciben la petición, llaman al servicio y devuelven JSON	No tienen reglas ni consultas

2.3 Conexión con la base de datos

La conexión se abre una sola vez, al arrancar la aplicación, en `app/extensions.py`:

```
def init_db(app):
    global _client, _db
    _client = MongoClient(app.config["MONGO_URI"])
    _db = _client[app.config["MONGO_DB_NAME"]]
    return _db
```

- La dirección de la base (`MONGO_URI`) y su nombre (`MONGO_DB_NAME`) se leen de variables de entorno, en `app/config.py`. No están escritas en el código.
- Toda la aplicación comparte ese único cliente. `pymongo` administra por dentro un grupo de conexiones y las reutiliza entre peticiones.
- La base se entrega a cada repositorio por su constructor, y los repositorios a los servicios. Ningún servicio abre conexiones por su cuenta.
- En Docker, la aplicación llega a la base por el nombre del contenedor: `mongodb://mongo:27017`.
- `GET /health` responde si el servidor está arriba.

Cómo se comprueba que funciona. El proyecto tiene 484 pruebas automáticas (`python -m pytest`) que ejercitan los modelos, los servicios y las rutas sobre una base en memoria, con los mismos índices de la base real.

2.4 Qué funciona hoy

Rol	Puede
Todos	Iniciar y cerrar sesión; cada rol llega a su propia pantalla
Superadmin	Registrar empresas con sus límites de usuarios y su logo, y crear sus Administradores
Administrador	Crear usuarios y actividades; asignar a uno o varios operarios; ver el registro y la evidencia de cada operario; ver la carga del equipo; crear turnos y validar la asistencia

Rol	Puede
Operario	Marcar su entrada al llegar y su salida al terminar; ver su día como una línea de tiempo; iniciar, pausar con motivo, reanudar y finalizar actividades, con evidencia (observación, fotos y adjuntos); ordenar sus actividades de horario flexible

---

2.5 Las reglas viven en las clases

Las reglas de negocio no dependen de que la pantalla esconda un botón: cada clase protege sus propios cambios de estado. Si se pide algo que el estado no permite, el método lanza un error antes de cambiar nada (una **precondición**), y la base de datos nunca queda en un estado inválido.

Clase	Método	Regla que protege
`Actividad`	`cancelar()`	No se cancela lo que ya terminó ni lo que alguien tiene en curso
`Ejecucion`	`pausar()`, `reanudar()`, `finalizar()`	Solo se pausa lo que está en curso, solo se reanuda lo pausado y solo se finaliza lo que está en curso
`Asistencia`	`corregir()`	Una corrección exige motivo y conserva las marcas originales
`Usuario`	`desactivar()`, `reactivar()`	No se desactiva dos veces ni se reactiva a quien ya está activo
`Evidencia`	`desde_archivo()`	Solo fotos o PDF, de 5 MB como máximo, reconocidos por su contenido

Las reglas que dependen de otros datos (por ejemplo, "una actividad que alguien ya inició no se edita" o "un operario con actividades sin terminar no se desactiva") las aplica el servicio, que consulta los repositorios. En el alcance hay una tabla con lo que se permite en cada estado de la actividad, y cada regla tiene su prueba automática.

3. Interfaz del sistema

El diseño pasó por tres momentos, y de cada uno hay evidencia en el repositorio.

Momento	Qué es	Dónde está
Bosquejos y maquetas	Composición de cada pantalla antes de programarla	`docs/disenos/`
Exploración con una herramienta de diseño	Propuesta generada en Google Stitch y su revisión crítica	`docs/disenos/stitch_calma_operativa/`
Prototipo funcional	La interfaz real, conectada a la API y a la base	`app/static/`

Maquetas

Archivo	Pantalla
`2_admin_equipo_tablero.png`	Administrador: tablero del turno, una fila por persona
`3_admin_proyecto.png`	Administrador: detalle de un proyecto
`4_operario_fila_del_tablero.png`	Operario: su día, actividad urgente, detalle y devolver
`5_operario_ordenar_dia.png`	Operario: hora fija, horario flexible y ordenar el día
`6_operario_hora_fija_parte.png`	Operario: actividad interrumpida por una de hora fija

Decisiones de diseño

- **Metáfora central:** el tablero de programación del turno que se usa en planta. El Administrador ve el tablero completo y el Operario ve su propia fila, en vertical.
- **Sistema visual:** colores, tipografía y espacios definidos como variables en un solo archivo (`app/static/css/estilos.css`), para que todas las pantallas se vean iguales.
- **Estados:** siempre con color, ícono y texto, para no depender solo del color.

Del bosquejo al prototipo funcional

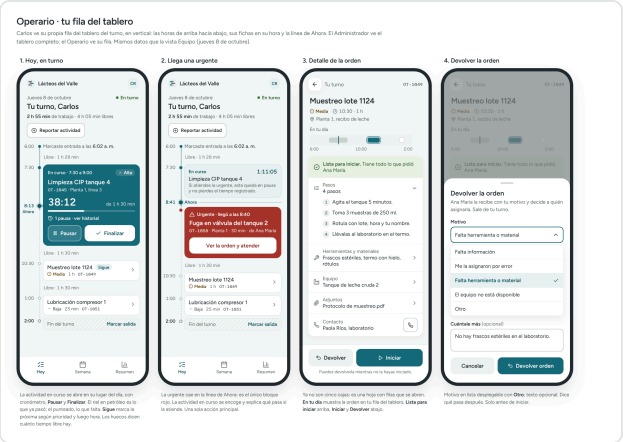


Figura 1. Maqueta de la pantalla Hoy del Operario.



Figura 2. La misma pantalla, ya construida y con datos reales.

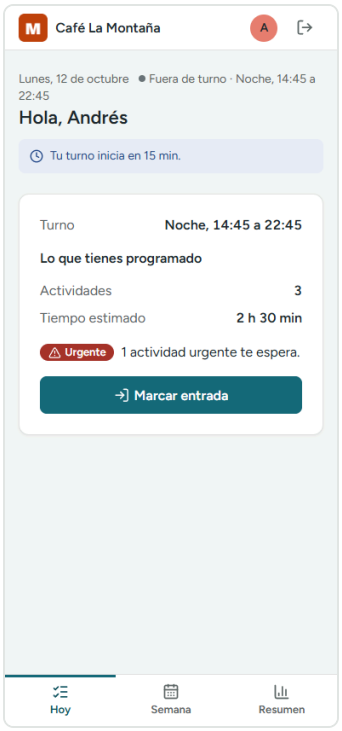


Figura 3. Operario: inicio de la jornada.

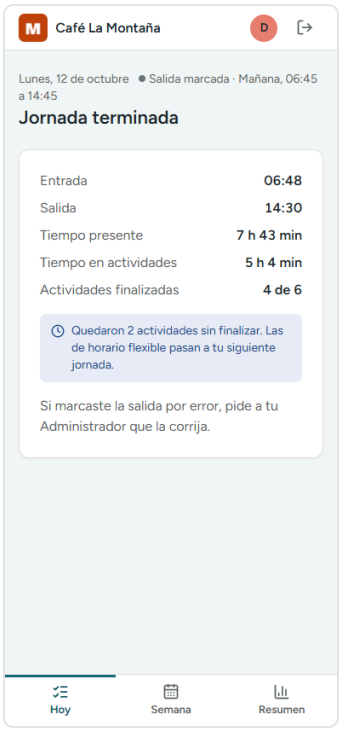


Figura 4. Operario: cierre de la jornada.



Figura 5. Inicio de sesión.

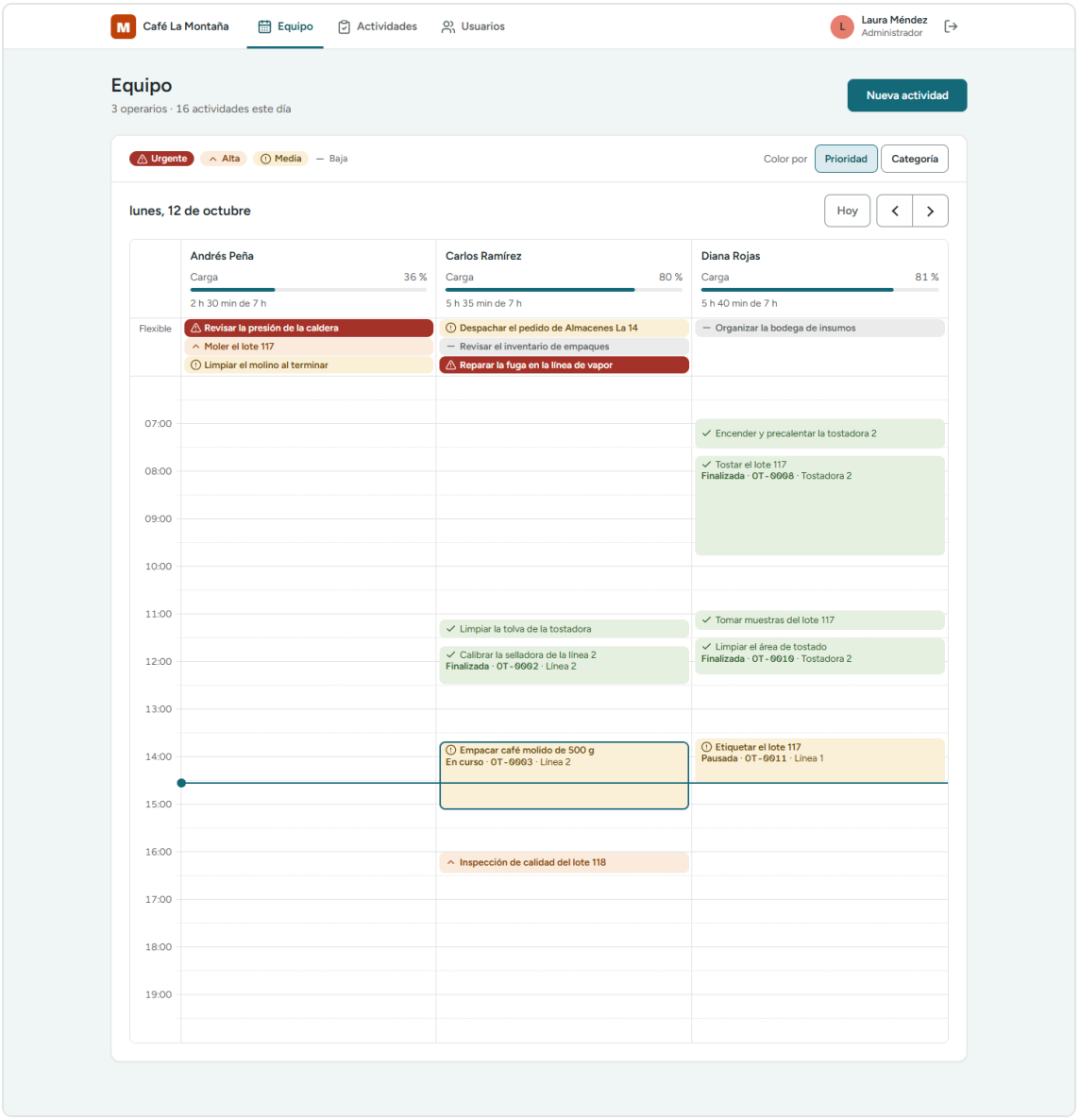


Figura 6. Administrador: horario del equipo, con la carga de cada operario.

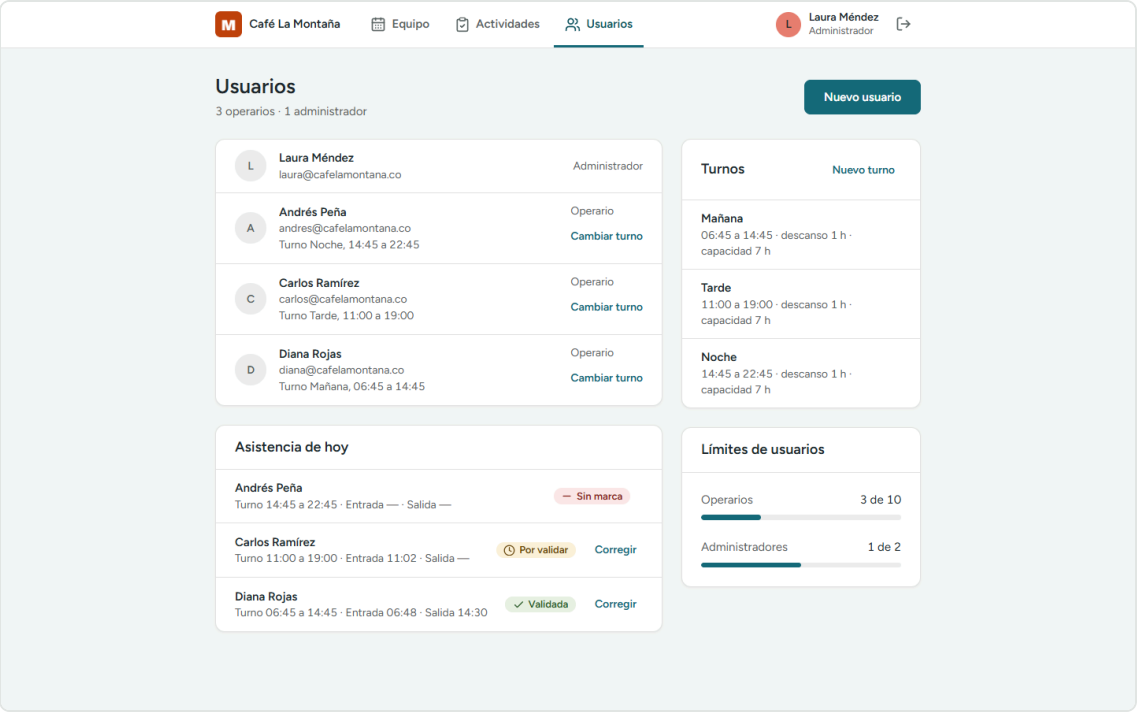


Figura 7. Administrador: usuarios, turnos y asistencia del día.

Las capturas se tomaron de la aplicación corriendo, con datos de demostración. El diseño de la interfaz se documenta en detalle en el trabajo de Diseño de Interfaces.

---

4. Organización del equipo

Pendiente: completar con los datos del equipo.

Integrante	Rol	Responsabilidades	Actividades a cargo

Forma de trabajo

- Control de versiones con Git y GitHub. Cada funcionalidad se hace en una rama propia y se integra por Pull Request.
- Antes de integrar, el código debe pasar las pruebas automáticas y el análisis de SonarQube.
- Las reglas del sistema se escriben primero en el documento de alcance; el código sigue al alcance.

---

5. Socialización del avance

5.1 Qué se ha desarrollado

Criterio del producto mínimo	Estado
Empresas y usuarios con tres roles	Terminado
Asignación y prioridad	Terminado
Actividades y proyectos	Parcial: las actividades funcionan completas; los proyectos ya están en la API y falta su pantalla
Ejecución (iniciar, pausar, reanudar, finalizar)	Terminado: las cuatro acciones en pantalla, con motivo de pausa y evidencia al finalizar
Registro de tiempo	Parcial: el servidor lo calcula; falta compararlo en pantalla con el estimado
Análisis de carga laboral	Parcial: carga por jornada y alerta al superar el 100 %
Turnos y asistencia	Parcial: el operario marca entrada y salida y el Administrador las valida; falta el aviso de posible ausencia
Indicadores de cumplimiento	Pendiente

5.2 Qué falta por completar

1. Pantalla del Operario: aviso de hora fija y devolver una actividad.
2. Pantallas del Administrador: tablero del turno y proyectos; retirar los requerimientos.
3. Indicadores de cumplimiento.
4. Aviso de posible ausencia y reasignación por ausencia.

5.3 Dificultades encontradas

- **De quién es la ejecución.** Al permitir varios operarios en una actividad, no estaba claro si el tiempo era de la actividad o de cada persona. Se decidió que la ejecución pertenece a la asignación, y eso cambió el modelo de datos.
- **Peticiones simultáneas.** Validar en el código "este operario no tiene nada en curso" no basta si llegan dos peticiones a la vez. Se resolvió con índices únicos en la base.
- **Cambiar el modelo con datos ya guardados.** Pasar de requerimientos a proyectos exigió un script de migración (`herramientas/migrar_etapa4.py`) para no perder la información existente.
- **Turnos de noche.** Un turno que empieza un día y termina al siguiente obligó a definir la jornada como el día en que inicia el turno.
- **La hora de los eventos.** Para que los tiempos sean confiables, la hora la pone siempre el servidor, nunca el navegador.
- **Borrar o conservar.** Borrar una actividad, un usuario o una empresa dejaba registros sin dueño. Se decidió que nada se borra: las actividades se cancelan y los usuarios y las empresas se desactivan.

- **Archivos que suben los usuarios.** Un archivo puede decir que es una foto y no serlo. Por eso el tipo se reconoce por sus primeros bytes, se limita su tamaño y la cantidad, y las fotos se reducen en el celular antes de subirlas.